

File Deduplication

Pavlov Aleksandr s2400691

1 ADT Design

1.1 First Level: Size Table

The core structure is a **Hashmap** keyed by file size (`long`). Each value in this hashmap is a pointer to a **FilePaths** structure, which groups files that share the same size.

1.2 Second Level: FilePaths

FilePaths manages files of the same size:

- **Single file**: when only one file has been registered for a given size, the structure stores just its path pointer. No hashing or comparison is needed.
- **Several files**: upon receiving a second file of the same size, the structure transitions to a hashmap keyed by file hash (via tabulation hashing). Each value in this hashmap is a pointer to a **Hashset** of path strings that share both the same size and the same content.

1.3 Structures

The **Hashmap**, **Hashset**, and **Vector** are generic collections. The hashmap stores key-value pairs in a **Vector**, with a separate index table to entry indices.

2 Algorithms

2.1 FDCheck: Adding a File

When a file is added, its size is computed (`file.size`) and used as the key to store the file in a **Hashmap**. If another file with the same size is added, a new **Hashmap** is created that indexes files by their content. For each file, a hash is computed (`file_hash_by_path_ptr`) and used for fast lookup in the table; in case of hash collisions, a byte-by-byte comparison of the files is performed (`file_equal_by_path_ptr`). The value in this map is a **Hashset** that stores file paths, also using a hash (`string_hash_by_ptr`) derived from the string and performing string comparison (`string_equal_by_ptr`) when necessary.

2.2 Tabulation Hashing

Files and strings are hashed using tabulation hashing with a table of 32×256 random 64-bit (`size_t`) values. The table is initialized once on the first call to the hash function.

2.3 FDDump: Retrieving Results

To obtain the results, we iterate over all hashmaps and compute the total number of entries. Then we allocate a table of the required size and iterate again to populate it.

3 Complexity Analysis

N number of files, S size of a file in bytes, d number of distinct file sizes, f number of files sharing a given size, h number of files sharing the same size and hash.

3.1 Time Complexity of FDCheck

Case	Average	Worst case
New size (no collision)	$O(1)$	$O(d)$
Same size, different hash	$O(S)$	$O(S + f)$
Same size, same hash	$O(S \cdot h)$	$O(S \cdot h + f)$

New size: In the average case this is $O(1)$. The worst case $O(d)$ occurs when the size table's hashmap triggers a reallocation and rehash of all d existing elements.

Same size, different hash: Computing the hash, cost $O(S)$. The hash resolves to a new place in the content hashmap. The worst case $O(f)$ for a possible reallocation of the hashmap.

Same size, same hash: the content hash collides with h existing entries. Each requires a byte-by-byte file comparison, $O(S \cdot h)$. The worst case adds $O(f)$ for reallocation.

Overall worst case: if all files have the same size and their hashes collide, the total cost for N files is $O(N^2 \cdot S)$.

3.2 Other Operations

Operation	Average	Worst case
FDInit	$O(1)$	$O(1)$
FDFree	$O(N)$	$O(N)$
FDDump	$O(N)$	$O(N)$

3.3 Space Complexity

The ADT stores $O(N)$ path copies and hashmap entries. Peak heap usage is $O(N \cdot L)$, where L is the average path length.

4 Performance Study and Testing

4.1 Benchmark Setup

Three scenarios are benchmarked, each time averaged over 10 runs. Both wall-clock time (`timespec.get`) and CPU time (`clock()`) are measured. File sizes tested: 1 MB, 2 MB. File counts: 100, 200, 400, 600, 800, 1000.

4.2 Analysis

Table 1: 1 MB files

Scenario	100		200		400		600		800		1000	
	wall	cpu	wall	cpu	wall	cpu	wall	cpu	wall	cpu	wall	cpu
Different sizes	0.012	0.011	0.024	0.022	0.049	0.047	0.071	0.072	0.095	0.087	0.118	0.106
Unique content	0.096	0.080	0.189	0.183	0.383	0.356	0.573	0.573	0.769	0.770	0.978	0.975
Identical content	0.143	0.144	0.286	0.286	0.580	0.580	0.859	0.833	1.143	1.097	1.435	1.428

Table 2: 2 MB files

Scenario	100		200		400		600		800		1000	
	wall	cpu	wall	cpu	wall	cpu	wall	cpu	wall	cpu	wall	cpu
Different sizes	0.012	0.011	0.023	0.023	0.047	0.047	0.070	0.070	0.095	0.095	0.118	0.119
Unique content	0.170	0.170	0.344	0.344	0.702	0.700	1.069	1.026	1.428	1.405	1.811	1.798
Identical content	0.250	0.248	0.503	0.503	1.023	1.006	1.520	1.506	2.062	2.062	2.555	2.525

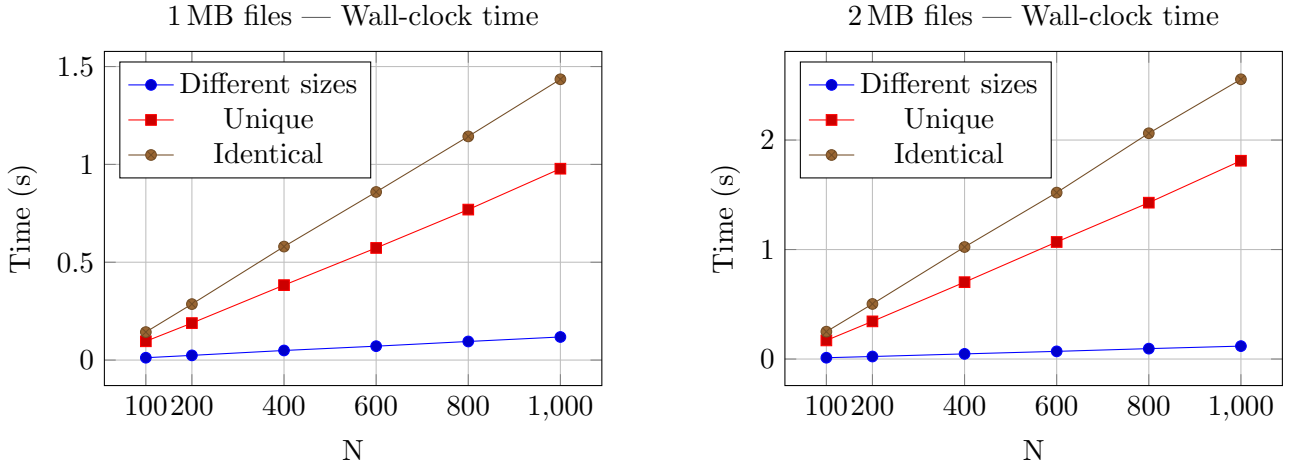


Figure 1: Wall-clock time vs number of files

Linear scaling in N . All scenarios show linear growth. For unique files at 1 MB: 0.096s at $N=100$ scales to 0.978s at $N=1000$ ($9.8\times$ increase from a $10\times$ increase in N), confirming $O(N)$ total complexity.

Different sizes is constant per file. The different sizes row is nearly independent of file size (0.118s for both 1 MB and 2 MB at $N=1000$), because file content is never read—only `file_size` is called.

Linear scaling in S . Comparing unique content at $N=1000$: 0.978s for 1 MB vs 1.811s for 2 MB—a factor of $\approx 1.85\times$, close to the expected $2\times$ for a $2\times$ increase in file size. This confirms $O(S)$ per-file cost when content hashing is required.

Identical files are slower than unique. Duplicates require both hashing and byte-by-byte comparison to confirm identity. At $N=1000$ with 1 MB files, identical content takes 1.435s vs 0.978s for unique—a factor of $\approx 1.47\times$, consistent with the additional file reads needed for equality verification.

4.3 Testing

Unit tests were written for three modules: **Vector**, **Hashmap** and **FILEDEDUP**.

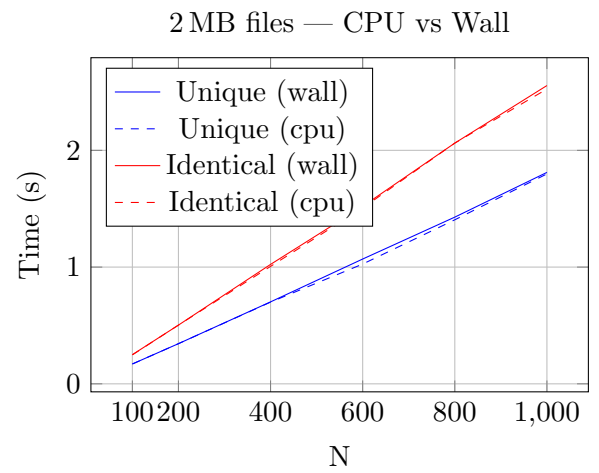
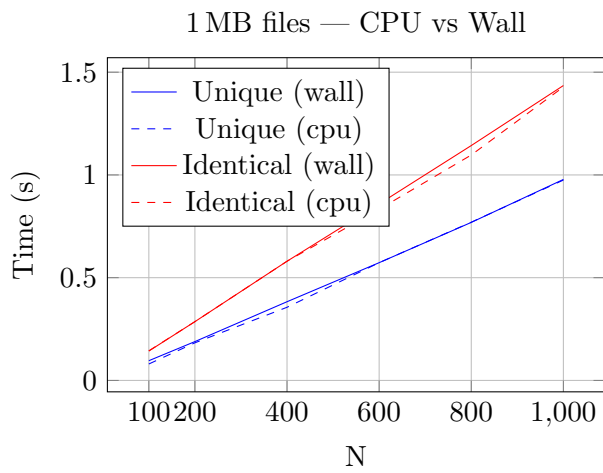


Figure 2: CPU time vs Wall-clock time comparison